

SEVERANCE: An Air-Gapped Local-LLM Document Workbench in Which Deterministic Code, Not the Model, Enforces Access, Citations and Egress

[Author Name 1], [Author Name 2], [Author Name 3], [Author Name 4]

Team Blunder — [Department], [Institution], [City], India

[corresponding-author@institution.ac.in]

Abstract—Refineries, public-sector undertakings, defence units and government offices hold large volumes of manuals, procedures and audit records, yet many of them cannot use cloud AI assistants because their documents are classified and outbound traffic is prohibited. Running an open-weight model locally removes the cloud, but not three remaining problems: the model can reveal text the user is not cleared to read, it can state facts or cite pages that are not in the source, and the claim that no data leaves the premises is usually asserted rather than shown. We present SEVERANCE, a self-hosted workbench built on one rule: the model proposes, deterministic code disposes. Access is decided by a two-axis label model (hierarchical grade and need-to-know compartments) enforced at the retrieval boundary, so denied text never reaches the prompt. Every citation must be a verbatim span of the exact document page it names; answer sentences the passages do not support are removed; a draft that cannot be verified is retried with the precise failure reason and otherwise withheld. A loopback-only guard checks every outbound call before a socket is opened, and generated code runs in an operating-system sandbox with no network access. On an 8 GB Apple M2 laptop with 3-billion-parameter local models and a 76-page petroleum-sector regulation, the verifier accepted all 810 genuine quotes and rejected all 2,701 quotes with substituted words, inserted negations, wrong documents or shuffled word order; the access gate agreed with a reference specification on all 102,400 grade-compartment-label combinations and returned no unreadable passage in 3,072 gated retrievals; the audit chain detected and localised all 200 simulated edits to non-final records. In a live run of 28 queries, 14 of 16 document questions were answered with verified citations and 2 were withheld, and every problem traced to the lexical retriever missing the page that held the answer; all 99 outbound calls went to the local model server and the only blocked call was a deliberate external probe. We also report the system's failure modes and limits.

Index Terms—*air-gapped AI, retrieval-augmented generation, citation verification, mandatory access control, local large language models, data sovereignty, tamper-evident audit logs.*

1. Introduction

Engineers and officers in regulated organisations spend a large share of their time locating the right clause in long documents: emergency-response plans, standard operating procedures, inspection records, procurement audits and statutory manuals. Large language models (LLMs) combined with retrieval [1] are well suited to this work, but most deployments send documents to a cloud provider. For a refinery, a defence establishment or a government department that is frequently not permitted. In India, public authorities additionally operate under the Right to Information Act, whose Section 10 requires that non-exempt parts of a record be released after exempt portions are severed [2] — an obligation that makes *passage-level* access decisions, not just document-level ones, a practical requirement.

Open-weight models served locally (for example through Ollama [3]) remove the dependency on a cloud, and 3-billion-parameter models now run acceptably on ordinary laptops. Local inference alone, however, leaves three gaps:

- **Confidentiality inside the organisation.** Many retrieval-augmented systems filter results after retrieval or instruct the model not to disclose restricted content. A model that has been shown a restricted passage can still repeat it, and text inside documents can itself try to manipulate the model [4].
- **Unfaithful answers and citations.** LLMs produce fluent statements that the source does not support and attribute quotations to the wrong page or document [5], [6], [7]. Smaller models, the ones that fit on local hardware, do this more often.

- **Unverifiable sovereignty.** “No data leaves the premises” is normally a statement in a slide deck. Auditors need evidence: which connections were attempted, to where, and which were refused.

SEVERANCE addresses these gaps by moving every trust decision out of the model and into small, deterministic, testable code. The model is treated as an untrusted component whose output is a *proposal*: it may suggest an answer and the quotations that support it, but code decides what the model is allowed to see, whether each quotation really exists on the page it names, whether the answer is released, how the answer is classified, and whether any network connection may be opened. Our contributions are:

1. An architecture for an air-gapped document assistant in which access control, citation acceptance, output classification, audit logging and network egress are all enforced by deterministic code, with the LLM confined to drafting and classification.
2. Enforcement of a two-axis (grade and compartment) label model at the retrieval boundary, so that denied passages are discarded before prompt assembly, combined with high-water-mark classification of every released answer.
3. A verification loop consisting of a verbatim span verifier bound to document *and* page, a sentence-level lexical grounding filter, retry with precise machine-generated feedback, and abstention when verification fails.
4. Enforced rather than asserted egress control: a loopback-only guard on every outbound call of the application, an operating-system sandbox for generated code, and a live, exportable network log with an on-demand blocked-probe test.
5. An evaluation on commodity hardware covering adversarial citation perturbations, exhaustive access-gate checking, ledger tampering, sandbox escape attempts and a live end-to-end run with a real regulatory document, including an honest account of failures.

Section 2 reviews related work, Section 3 defines the threat model, Section 4 describes the design, Section 5 the implementation, Section 6 the evaluation, and Section 7 discusses limitations.

2. Background and Related Work

2.1 Retrieval augmentation and attribution

Retrieval-augmented generation (RAG) conditions a generator on retrieved passages to improve factuality [1]. Hallucination nevertheless persists [5]. Work on attribution evaluates whether generated text is supported by cited sources: ALCE benchmarks citation quality using natural-language-inference (NLI) models [6], AIS formalises attribution to identified sources [7], FActScore scores atomic facts against a knowledge source [8], and RAGAS provides reference-free RAG metrics [9]. These approaches *measure* support with learned models, usually offline. SEVERANCE instead *enforces* a narrower property at answer time with deterministic code: each quotation must literally occur on the page it cites, and each answer sentence must be lexically supported by the passages. The property is weaker than entailment, but it is exact, explainable and cannot be argued with by the model; Section 7 discusses what it does not catch.

2.2 Access control

Multi-level security with hierarchical levels and non-hierarchical categories goes back to the Bell–LaPadula model [10] and Denning's lattice model of information flow [11]; Saltzer and Schroeder's principles of fail-safe defaults and complete mediation [12] and role-based access control [13] shape most enterprise systems. Labelling an output with the highest classification of its inputs is the classic high-water-mark policy [14]. SEVERANCE does not propose a new access-control model. Its contribution is *where* the check is applied: between scoring and prompt assembly, so that the text of a denied passage never exists downstream of the gate, and the released answer inherits the label of everything the model saw.

2.3 Tool use and iterative agents

ReAct interleaves reasoning with actions [15], Toolformer teaches models to call APIs [16], and Reflexion feeds verbal feedback back into later attempts [17]; functional correctness of generated code is commonly measured by executing it [18]. SEVERANCE uses a constrained form of each: the model only extracts arguments for deterministic tools (so arithmetic is never done by the model), and generated code is executed in a sandbox whose real error output drives up to three repair attempts. The same feedback principle drives citation retries.

2.4 Tamper-evident logging and OCR

Hash-chained logs make undetected modification of past records infeasible without rewriting all later records [19], [20]. SEVERANCE uses a simple SHA-256 chain; its main known weakness — rewriting the most recent record — is measured in Section 6. Scanned pages are handled with the Tesseract OCR engine [21].

3. Problem Setting and Threat Model

An organisation holds a corpus of documents; each document d carries a label consisting of a tier $t(d) \in \{\text{public, internal, confidential, secret}\}$ and a set of compartments $K(d) \subseteq \{\text{hse, vigilance, legal, commercial, technical}\}$. Each authenticated user u has a grade $g(u)$ from the organisation's pay-grade structure and a set of granted compartments $C(u)$. The system must answer questions from the corpus while satisfying the following requirements.

- **R1 — No read above clearance.** No text from a passage u may not read reaches the model, the answer, the report or the logs visible to u .
- **R2 — Grounded answers only.** Every released answer is supported by quotations that exist verbatim on the cited page; otherwise nothing is released.
- **R3 — No external egress.** The application makes no network connection to any non-loopback host, including through code it generates and runs.
- **R4 — Accountability.** Every query and clearance change is recorded, and later modification of a record is detectable.

Threats in scope. (T1) An authenticated user, possibly very senior, asks questions whose answer lies in documents outside his or her compartments. (T2) The model hallucinates facts or misattributes quotations. (T3) The application, or code produced by the model, attempts to contact an external host. (T4) Someone with write access to the database edits a past audit record. (T5) Text inside a document attempts to steer the model into disclosing other content; because denied text is never placed in the prompt, the model cannot disclose what it was never given.

Out of scope. A compromised operating system or administrator account, physical attacks and side channels, backdoored model weights, and an attacker able to rewrite the entire audit database. The egress guard is an application-level control; Section 7 explains why it should be paired with a host firewall.

4. System Design

Figure 1 shows the request path. Dark boxes are deterministic code that makes decisions; orange boxes are local LLM calls whose output is only a proposal.

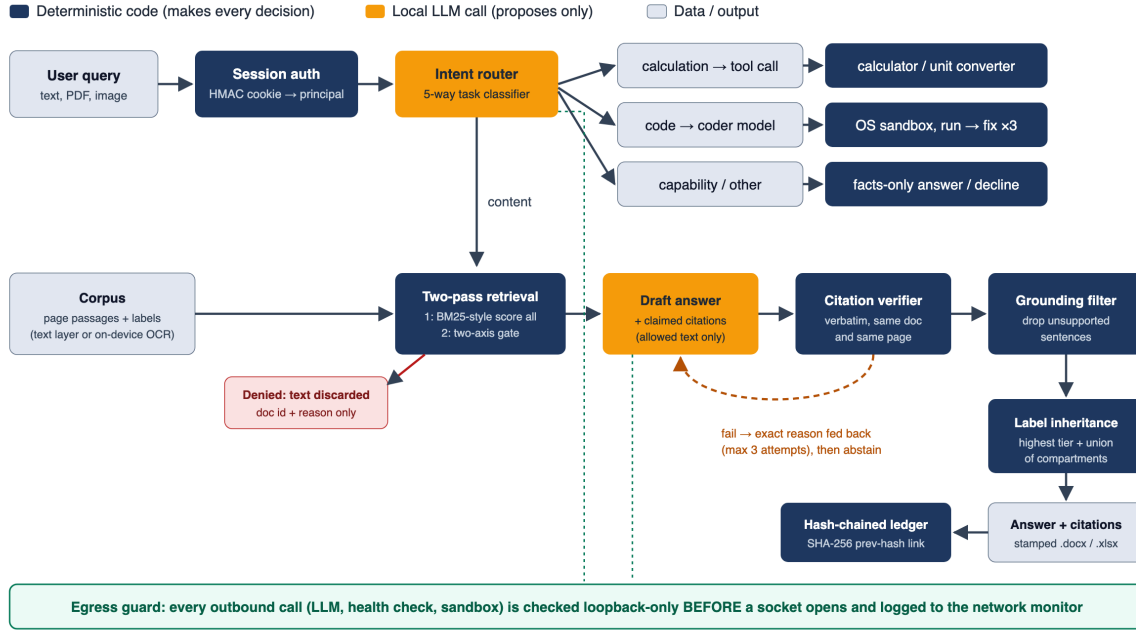


Fig. 1. SEVERANCE request path. The router sends each question to one of five paths; the document path passes through a two-axis gate, drafting, verbatim citation verification and a grounding filter, with retry and abstention. All outbound calls pass a loopback-only egress guard.

4.1 Two-axis access model

Grades are mapped to a rank ladder (non-management grades S1–S4, TS1–TS6 and JM1–JM6 to ranks 1–6; officer grades A–I to ranks 7–15). Each tier has a minimum rank: public 0, internal 1, confidential 7 and secret 11. A user may read a passage if and only if

$$\text{read}(u, d) \Leftrightarrow \text{rank}(g(u)) \geq \text{floor}(t(d)) \wedge K(d) \subseteq C(u)$$

The two conditions are independent: seniority never substitutes for a compartment. A general manager (rank 12) without the vigilance compartment cannot read a confidential vigilance report that an assistant manager (rank 7) holding that compartment can. The function is pure — no database, clock or network — and fails closed: an unknown grade, an unknown tier or a malformed input yields *deny*. Administrative powers are separated: administrators create accounts and set grades but cannot grant compartments; each compartment has a designated sponsor who may grant only that compartment and never to himself or herself; every change is written to the audit ledger.

A released answer is labelled with the highest tier and the union of compartments of *all* passages placed in the prompt (a high-water mark [14]), because the model may have been influenced by any of them. The label is stamped in three places on generated reports: a banner, the page header and footer, and the file name.

4.2 Two-pass retrieval with text discard

Retrieval scores every page of the corpus first, ignoring labels, using a BM25-style term-frequency saturation score over stop-word-filtered tokens [22]. Only then does it walk the ranked list through the gate. Readable passages are kept until k have been collected; for a denied passage only a *denial* is recorded — document identifier, the deterministic reason and the required label — and its text is dropped at that point. Denials are collapsed per document so users learn that a relevant document exists and why it is withheld, without learning its content. Scoring before gating is deliberate: the system must know a passage matched in order to report that it was withheld. Passages with zero lexical overlap are never passed on, and if no readable passage matches, the system abstains without calling the model.

4.3 Task routing and model selection

A classifier call at temperature 0 assigns each message to one of five categories — *content* (a question about the documents), *calculation*, *code*, *capability* (a question about the assistant itself) or *other* — and each category is routed to a different path and, where useful, a different model: an IBM Granite 3B model [23] for classification and drafting, Qwen2.5-Coder 3B [24] for code, Granite Vision [23] for images, and Llama 3.2 [25] as a drafting fallback. If classification fails, the request falls through to the content path, which is the most heavily checked.

4.4 Verified drafting

The drafting prompt contains only the readable passages, each preceded by a header naming its document and page, and asks for a JSON object with an answer and a list of citations. The verifier then applies, in order:

- at least one citation must be present (an answer with no citations is never treated as verified);
- each quotation must be 20–300 characters and at least five words long;
- the cited (document, page) pair must be one of the passages actually placed in the prompt;
- the quotation must occur in that page's text — first as an exact substring, then after collapsing whitespace, and finally after a lenient normalisation that removes amendment markers such as 72[...] and compares only the lowercase alphanumeric sequence.

The lenient stage exists because PDF extraction introduces line-wrap splits, footnote markers and typographic punctuation that a model naturally cleans up when it quotes; Section 6.2 measures how many harmless transcription variants it admits and whether it admits content changes. On failure the verifier returns a single precise reason (for example, that the quoted page was not in context and which pages were, when the model has mistaken a clause number such as “25.” for a page number). The reason is appended to the next prompt, up to three attempts; if all fail, the answer is withheld and the abstention is logged. When the model returns an answer with no citations, a narrower second call asks the same model only to extract supporting quotations for the answer it already wrote.

Verbatim quotations do not guarantee that the surrounding prose is supported, so a second deterministic filter scores each answer sentence. A sentence is removed when fewer than 70% of its content words (after stop-word removal and light stemming) occur in the passages, when it states a number greater than ten that appears in none of them, or when the source sentence it is built from has the opposite polarity (for example “do not use water” versus “use water”). The threshold was chosen from stored answers on this corpus, where grounded sentences scored 71–100% and invented ones 40–61%. Removed sentences are replaced by a visible note; if every sentence is removed, the attempt counts as failed.

4.5 Deterministic tools and sandboxed code

For calculations the model's only task is to emit a tool name and arguments; a calculator evaluates arithmetic by walking a restricted Python abstract syntax tree, and a unit converter handles pressure, temperature, volume (including the 158.987-litre petroleum barrel), length and mass from fixed tables. Invalid tool calls fall back to the code path. For code, the coder model writes a program, the sandbox runs it, and any error output is fed back for up to three attempts; the final state — passed first time, recovered after a retry, or failed — is reported as it is. Each run uses a fresh isolated interpreter (python -I), a wall-clock timeout, POSIX limits on CPU time, address space, file size and process count, and an empty temporary directory. On macOS the process additionally runs under a sandbox-exec profile in which the kernel refuses every network operation and every write outside the run's directories; on other platforms name resolution and connection calls are blocked at the Python level.

4.6 Multimodal ingestion

Pages are extracted with their text layer; a page whose text layer is empty is rasterised with PyMuPDF and passed to Tesseract [21]. OCR output flows through exactly the same verifier, so a citation to a scanned page must match

the OCR text. Ad-hoc uploads (images and slide decks) are held in memory, scoped to the uploading user and never added to the governed corpus. Image descriptions from the vision model cannot be checked against a source string; they are shown under a separate heading that states they are model observations, not verified findings.

4.7 Audit ledger

Every answered or withheld query, login and clearance change is appended to a ledger whose rows store the SHA-256 hash of a canonical JSON encoding of the row (timestamp, actor, action, details and previous hash); the first row links to a hash of 64 zeros. Verification recomputes the chain. The ledger is readable by every authenticated user as a transparency log, so it stores only a short answer preview; full answers are stored separately and are returned only to the user who asked or to an administrator.

4.8 Egress guard and network monitor

Every outbound HTTP call the application makes — model inference, model-server health checks and vision analysis — first calls a guard with a purpose label and the destination URL. The guard parses the host and allows it only if it is a loopback literal, the name “localhost”, or a name that resolves to a loopback address (a name that fails to resolve is treated as external). A non-loopback destination raises an exception *before* any socket is opened, and both outcomes are written to a network log. The build contains no cloud model client. A monitoring view shows live totals, the per-call stream with purpose labels, filtering and JSON export, and a *test probe* that deliberately attempts to reach a real external address through the same guard so that an auditor can watch it being refused.

4.9 Deliverables

Verified answers can be exported as Word and Excel reports carrying the inherited classification, the verified citations and the list of withheld documents. Reports are refused for withheld answers, and download is restricted to the user who asked the question or an administrator.

5. Implementation

SEVERANCE is written in Python 3.12 with FastAPI, Pydantic data contracts and SQLite, and uses Ollama as the local model server. The application comprises about 7,400 lines of Python and a browser interface in plain HTML, Tailwind CSS and JavaScript with light and dark themes. Passwords are hashed with scrypt and sessions are carried in HMAC-SHA256-signed, HTTP-only cookies. The repository contains 108 automated tests (about 2,100 lines) covering the access gate, sponsor rules, retrieval, verification, grounding, ledger, sandbox, tools, report generation and API behaviour; all pass. No GPU is required.

6. Evaluation

6.1 Setup

All experiments ran on an Apple M2 laptop with 8 GB of unified memory running macOS 26.6, with no GPU acceleration beyond the integrated one used by Ollama. Models: granite4.1:3b (classification and drafting), qwen2.5-coder:3b (code), llama3.2:latest (drafting fallback) and granite3.2-vision (vision). The corpus has four documents, 82 pages and 137,650 characters. Seventy-six pages are a real public regulation: the Petroleum and Natural Gas Regulatory Board's Codes of Practices for Emergency Response and Disaster Management Plan (ERDMP) Regulations, 2010 [26], containing amendment markers, tables, checklists and forms. The remaining three documents are short synthetic documents (six pages in total) standing in for internal HSE, vigilance and RTI records; they carry different tiers and compartments so that the access model can be exercised. No real confidential documents were used. Experiments E1–E4 are deterministic and involve no model; E5 and E6 use the live models. Scripts and raw outputs are available from the authors [repository link].

6.2 E1 — Robustness of the citation verifier

For every page we sampled up to ten genuine quotations (random word windows of 6–25 words satisfying the length rules), giving 810 quotations, and derived perturbed versions of each. Every quotation was checked against all pages of its document, as in a multi-page prompt. Table I reports how many were accepted.

TABLE I. Acceptance of genuine and perturbed quotations by the verifier (seeded random sample).

Quotation type	Should be	n	Accepted	Rate
Genuine span	accepted	810	810	100.0%
One word replaced	rejected	809	0	0.0%
“not” inserted after a modal verb	rejected	272	0	0.0%
Attributed to another document	rejected	810	0	0.0%
Words shuffled	rejected	810	0	0.0%
Attributed to another page, same document	rejected	810	2	0.2%
One interior word deleted	rejected	774	8	1.0%
Two adjacent words swapped	rejected	774	23	3.0%
First number changed	rejected	399	43	10.8%

All genuine quotations were accepted, and none of the 2,701 quotations with a replaced word, an inserted negation, a wrong document or shuffled words was accepted. We inspected every accepted case in the remaining rows. **All 43 accepted number changes** altered an amendment marker such as 134[— digits the verifier removes by design — and none altered a number in the regulation's content. **All 23 accepted swaps** and 4 of the 8 accepted deletions touched punctuation-only tokens (for example a list dash), leaving the sequence of words unchanged. Of the other 4 deletions, 3 removed a word next to such a token so that the result was still a genuine, shorter span of the page, and 1 was a boundary artefact of whitespace-insensitive matching: “(e) SIC shall also ensure...” matched “the SIC shall also ensure...” because the letters “e” and “SIC” are adjacent once spaces are removed; the words asserted were nevertheless present on the page. The 2 accepted wrong-page attributions were quotations that genuinely occur on both pages (a running title and a repeated footnote). No accepted quotation introduced a word, number or negation absent from the cited page.

TABLE II. Benign transcription variants: acceptance by SEVERANCE versus a plain exact-substring check.

Variant	n	SEVERANCE	Exact substring only
Line break inserted	810	100.0%	0.0%
Lower-cased	640	100.0%	0.0%
Typographic quotes and dashes	169	100.0%	0.0%
Amendment marker removed	54	100.0%	0.0%

Table II shows why normalisation is needed: a plain substring test would reject every one of these faithful transcriptions and force needless retries or abstentions, while the normalised comparison accepts them without admitting any of the content changes in Table I.

6.3 E2 — Access gate and retrieval boundary

We evaluated the gate on every combination of the 25 valid grades, all 32 subsets of the five compartments held by the user, the four tiers, and all 32 compartment subsets on the label — 102,400 decisions — and compared each with an independently written reference predicate. There were **0 disagreements**. In 49,984 of these combinations the user's rank met the tier floor but a compartment was missing; all were denied, confirming that seniority never substitutes for need-to-know. The headline inversion case behaves as intended: a general manager (grade F) holding only the commercial compartment is denied a confidential vigilance report, while an assistant manager

(grade A) holding vigilance is allowed. An unknown grade is rejected by the input schema, and if the schema is bypassed the gate still denies at every tier; a non-principal object is also denied.

To check the boundary rather than the predicate alone, we ran 3,072 retrievals ($16 \text{ queries} \times 6 \text{ grades} \times 32 \text{ compartment sets}$, $k = 5$). They returned 8,064 passages and 2,624 denials; **0 returned passages were unreadable** by the requesting user, and denial records carry no text field by construction.

6.4 E3 — Ledger tamper detection

We appended 500 records and verified the chain in 2.96 ms. We then simulated two attackers with direct database access on 100 randomly chosen rows each (Table III).

TABLE III. Detection of audit-record tampering.

Attack	Trials	Detected	Located correctly
Edit a record's action and details	100	100	100 (at the edited row)
Edit a record and recompute its own hash	100	100	100 (at the following row)
Edit the most recent record and recompute its hash	1	0	—

All edits to non-final records were detected and located. As expected for an unanchored hash chain, rewriting the newest record together with its hash is not detectable, and neither would be truncation of the tail; Section 7 describes the mitigation.

6.5 E4 — Sandbox containment

TABLE IV. Behaviour of generated code under the sandbox (macOS kernel-enforced mode, 5 s timeout).

Probe	Expected	Observed
Normal program	runs	ran, printed 45
Infinite loop	killed	killed at timeout (5.01 s)
HTTP request to example.com	refused	refused by kernel; attempt to example.com:80 logged
Raw socket to an IP address	refused	refused by kernel; attempt to 93.184.216.34:80 logged
Write to /tmp outside run folder	refused	write refused by kernel; no file created

In every case the enforcement mode was os, meaning the refusal came from the operating-system kernel rather than from Python code that the generated program could bypass. Attempted destinations were still recorded in the network log.

6.6 E5 — Live end-to-end run

We issued 28 queries once each through the complete pipeline, as a user cleared for every document, with a fresh agent per query as in the web application: 16 questions about the corpus (12 about the ERDMP regulation and four about the synthetic documents), one question whose answer is not in the corpus, one off-topic question, four calculations, four programming requests and two questions about the assistant. For each document question we recorded in advance the page or pages that contain the answer. Results are from a single run; because the models sample stochastically, repeated runs will vary.

Routing. 27 of 28 queries were routed to the intended path. The one miss was the off-topic cricket question (O2), which the classifier sent to the document path instead of *other*. Because no page shared a content word with it, retrieval returned nothing and the system replied that no relevant documents were found without calling the drafting model, so the misroute was harmless: the document path fails safe.

Document questions. Table V lists every document question. 14 of 16 were answered and 2 withheld. 13 of the 14 answers passed verification on the first attempt, the rest after one or more retries driven by verifier feedback. 11 of 14 answers cited at least one of the pages we had marked in advance. The three answers that did not cite a

marked page differ in kind. C4 cited page 33, where the definition of a major incident begins (fire for more than 15 minutes, explosion, fatality, more than 500 man-hours lost, shutdown) before continuing on page 34; the answer is correct and our advance marking was incomplete. C6 cited page 36, which mentions the transport emergency (TREM) card only in passing; the fields listed in the answer are correct and come from Schedule IX on page 56, which was also retrieved, but the quotation itself does not support them. C11 is discussed below. The grounding filter checked 27 answer sentences and removed 3 from 3 answers. The median time to a verified answer was 13.4 s (range 9.0–59.1 s).

What limited the run was retrieval, not verification. Re-running the deterministic retriever shows that the page containing the answer was among the three retrieved passages for 13 of the 16 questions, and all 13 were answered. The three misses account for every problem in the run. For C1 (definition of an off-site emergency, page 3) and C12 (first-aid items for road transport, pages 53–54) the answer page was not retrieved. In C1 the model offered the same unverifiable quotation three times despite the feedback; in C12 its first draft passed citation verification but its only sentence failed the grounding check, and the two later drafts failed verification. Both were withheld, which is the intended outcome when the evidence is not in context. For C11 (objectives of preparing an ERDMP, pages 2 and 6) the retriever returned the page listing the objectives of the *emergency control organisation* instead. The model answered that related question with genuine quotations and both checks accepted it; the answer names what it describes, but it does not answer the question asked, and it carried the amendment markers “72” and “73” into its prose as if they were references.

TABLE V. Live results for document questions. ERDMP = regulation; HSE, VIG, RTI = synthetic documents. *Answer page retrieved*: a page marked in advance was among the three retrieved passages. *Sent. removed*: sentences deleted by the grounding filter.

ID	Question	Answer page retrieved	Outcome	Tries	Cited pages	Marked page cited	Sent. removed	Time (s)
C1	How do the regulations define an off site emergency?	no	withheld	3	—	—	0	68.9
C2	What are the responsibilities of the Chief Incident Controller?	yes	answered	1	ERDMP p. 20	yes	0	21.4
C3	What are the duties of the Site Incident Controller?	yes	answered	1	ERDMP p. 20	yes	0	17.7
C4	Which incidents must be reported as major incidents?	yes	answered	1	ERDMP p. 33	no	0	12.9
C5	What emergency control systems and facilities should an installation have?	yes	answered	2	ERDMP pp. 29, 30	yes	1	59.1
C6	What information does a transport emergency card contain?	yes	answered	1	ERDMP p. 36	no	0	32.4
C7	What are the overall objectives of an emergency control organization?	yes	answered	1	ERDMP p. 18	yes	0	13.3
C8	What should the termination of an emergency concentrate on?	yes	answered	1	ERDMP p. 35	yes	1	13.5
C9	What is the responsibility of the Recorder during an emergency?	yes	answered	1	ERDMP p. 25	yes	0	12.6
C10	How is individual risk per annum categorized?	yes	answered	1	ERDMP p. 11	yes	1	14.9
C11	What are the objectives of preparing an ERDMP?	no	answered	1	ERDMP p. 18	no	0	36.1
C12	What should the first aid box carried for road transport emergencies contain?	no	withheld	3	—	—	0	26.5
C13	Where must personnel assemble during a toxic gas release at CDU-2?	yes	answered	1	HSE p. 1	yes	0	13.3
C14	What minimum water flow rate must deluge systems on crude distillation heaters maintain?	yes	answered	1	HSE p. 2	yes	0	10.1
C15	What did the vigilance inquiry find about the heat exchanger maintenance bids?	yes	answered	1	VIG p. 1	yes	0	9.7
C16	What does Section 10 of the RTI Act allow regarding severability?	yes	answered	1	RTI p. 2	yes	0	9.0

Unanswerable and off-topic questions. Asked for MRPL's net profit for 2023–24, which no document contains, the system retrieved the synthetic MRPL pages because they share the word “MRPL”. The grounding filter removed three of the draft's four sentences, and the released answer stated only that the documents do not provide financial information, supported by a verified quotation; no figure was released. The off-topic cricket question (O2) received “No relevant documents found matching your query.”

Calculations. 4 of 4 results were numerically correct. The model chose the unit converter for the three conversions and the calculator for the arithmetic expression, and the tools returned 3625.943443 psi, 11370, 75.477729 barrels and 113 °F. Each answer states that it was computed by a deterministic tool, not by the model, and each took under 3 s.

Code. 4 of 4 programs ran successfully in the sandbox, 4 on the first attempt, so these tasks did not exercise the repair loop. We therefore ran a separate request designed to fail — “write python code that converts the string “abc” to an integer using int() and prints the result” — three times. In 3 of 3 trials the first program stopped with the real interpreter error ValueError: invalid literal for int() with base 10: ‘abc’, that error was fed back, and the second program handled it and ran successfully (11.2, 16.3, 16.5 s end to end).

Capability questions. Both were answered from true facts supplied by the application — the real document list filtered by the caller’s clearance — rather than from the model’s own knowledge; the answer to “which documents can I access?” listed the four documents with their classifications.

6.7 E6 — Network egress during the live run

The network log for the live run (Table VI) contains 100 entries. All 99 allowed calls went to the local model server on the loopback interface; 0 allowed calls went anywhere else. 1 call was blocked: the deliberate test probe to an external address, which the guard refused before opening a socket. No other call was blocked, which shows the application did not try to reach an external host during normal operation (0 non-probe blocks). The audit ledger for the run verified as intact.

TABLE VI. Outbound call attempts recorded during the live run, by purpose.

Purpose label	Destination	Result	Count
intent-classification	localhost:11434	allowed	28
ollama-health-check	localhost:11434	allowed	28
document-drafting	localhost:11434	allowed	25
citation-repair	localhost:11434	allowed	8
code-generation	localhost:11434	allowed	4
tool-call-extraction	localhost:11434	allowed	4
capability-answer	localhost:11434	allowed	2
sovereignty-test-probe	example.com:443	blocked	1

7. Discussion and Limitations

Verbatim is not entailment. A quotation can exist on the cited page without supporting the sentence it is attached to, and the lexical grounding filter cannot catch a false relationship expressed with words that do occur in the source (for example asserting that an item was *misused* when the source merely lists it). SEVERANCE guarantees that what is quoted is real and that answer sentences are built from source vocabulary, not that the answer is correct. A local NLI or entailment model could be added as a further check, as in attribution research [6], [7].

Withholding costs recall. With a 3B model on a laptop, some answerable questions are withheld and verified answers take tens of seconds. We consider this the right trade-off for regulated settings — a withheld answer is visible and can be escalated, a confident wrong one may not be noticed — but it is a cost. Larger local models and a GPU server would reduce both effects without changing any guarantee.

Lexical retrieval. Term-overlap scoring misses paraphrased questions. Because the gate operates on candidates after scoring, dense embeddings can be added without weakening R1.

The egress guard is application-level. It mediates every call made through the application’s model client and records the destination, but a compromised third-party library that opened sockets directly would bypass it. Deployments should combine it with a host firewall or a network namespace that allows only loopback; the guard’s value is the auditable, per-purpose record and the fail-before-connect behaviour, not a claim to replace network controls. Generated code is covered separately by the kernel sandbox.

Ledger anchoring. An unanchored hash chain cannot detect rewriting or truncation of its newest records (Table III). Periodically recording the head hash outside the database — on paper, in a separate signed log or with a second custodian — closes this gap cheaply.

Sandbox scope. The sandbox restricts network, writes, time and resources but not reads, and its strongest mode depends on macOS. A Linux deployment should use namespaces and seccomp or a disposable container per run.

Evaluation scope. The live evaluation is a single run of 28 queries written by the authors on one real regulation and three small synthetic documents, answer quality was not graded by independent domain experts, and we did not compare against cloud assistants, which the target organisations cannot use in any case. A larger study with real internal documents, expert graders and repeated runs is the main next step. Vision outputs are presented as unverified and were not evaluated.

8. Conclusion

SEVERANCE shows that an organisation that cannot use cloud AI can still give its staff a useful document assistant on ordinary hardware, provided the model is not trusted with decisions. Placing the access check before prompt assembly means restricted text is never available to leak; binding every quotation to a real span of a named page, filtering unsupported sentences and withholding what cannot be verified keeps answers traceable; and a guard that refuses non-loopback connections before they open, together with a kernel sandbox for generated code and a live network log, turns the sovereignty claim into something an auditor can check. Future work includes dense retrieval behind the gate, local entailment checking, anchored audit logs, Hindi and regional-language OCR, and a field evaluation with domain experts.

Acknowledgment

[Optional: acknowledge mentors, institution and the Smart India Hackathon problem-statement owner.]

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [2] Government of India, *The Right to Information Act, 2005* (Act No. 22 of 2005), Section 10: Severability.
- [3] Ollama, “Ollama: Get up and running with large language models locally,” software, <https://ollama.com>, accessed Sep. 2026.
- [4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, 2023.
- [6] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proc. EMNLP*, 2023.
- [7] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, “Measuring attribution in natural language generation models,” *Computational Linguistics*, vol. 49, no. 4, 2023.
- [8] S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, “FActScore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proc. EMNLP*, 2023.
- [9] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proc. EACL: System Demonstrations*, 2024.
- [10] D. E. Bell and L. J. LaPadula, “Secure computer systems: Mathematical foundations,” MITRE Corporation, Tech. Rep. MTR-2547, 1973.
- [11] D. E. Denning, “A lattice model of secure information flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [12] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [13] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

- [14] C. Weissman, “Security controls in the ADEPT-50 time-sharing system,” in *Proc. AFIPS Fall Joint Computer Conference*, 1969, pp. 119–133.
- [15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [16] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [18] M. Chen *et al.*, “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [19] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999.
- [20] S. A. Crosby and D. S. Wallach, “Efficient data structures for tamper-evident logging,” in *Proc. 18th USENIX Security Symposium*, 2009.
- [21] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, 2007.
- [22] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [23] IBM, “Granite model family (language and vision),” model cards and documentation, <https://www.ibm.com/granite>, accessed Sep. 2026.
- [24] B. Hui *et al.*, “Qwen2.5-Coder technical report,” arXiv:2409.12186, 2024.
- [25] A. Grattafiori *et al.*, “The Llama 3 herd of models,” arXiv:2407.21783, 2024.
- [26] Petroleum and Natural Gas Regulatory Board, *Codes of Practices for Emergency Response and Disaster Management Plan (ERDMP) Regulations, 2010*, notification G.S.R. 39(E), New Delhi, 18 Jan. 2010, as amended.